

SWAM: Event-Driven Incident Management System

Technical Presentation

Muhammad Sohail

Submitted to: Prof. Enrico Vicario



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.

Table of Contents

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ System Demonstration

Table of Contents

1 Introduction & Proposed System

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ System Demonstration
- ▶ Conclusions & Future Work

Objective and Purpose of the Project Work

1 Introduction & Proposed System

Scenario

Establish a highly responsive platform to manage incident creation. Issues must be dynamically tagged, routed to specific departments with exclusive ownership, and resolved through direct, collaborative comment threads.

Objective and Purpose of the Project Work

1 Introduction & Proposed System

Scenario

Establish a highly responsive platform to manage incident creation. Issues must be dynamically tagged, routed to specific departments with exclusive ownership, and resolved through direct, collaborative comment threads.

Current Limitation

Standard systems struggle when many people report the exact same issue at once. Trying to find and merge these duplicate reports while the user is waiting slows the whole application down and creates massive delays.

Objective and Purpose of the Project Work

1 Introduction & Proposed System

Scenario

Establish a highly responsive platform to manage incident creation. Issues must be dynamically tagged, routed to specific departments with exclusive ownership, and resolved through direct, collaborative comment threads.

Current Limitation

Standard systems struggle when many people report the exact same issue at once. Trying to find and merge these duplicate reports while the user is waiting slows the whole application down and creates massive delays.

Business Impact

Duplicate alerts are handled in the background, keeping incident logging instant. This sustains performance during traffic spikes, reduces alert clutter for IT admins, and accelerates resolution times.

Proposed System

1 Introduction & Proposed System

The application is built as three separated layers

Frontend

Angular Single Page Application (SPA)

Client-side rendering, communicating with the backend over REST.

Proposed System

1 Introduction & Proposed System

The application is built as three separated layers

Frontend

Angular Single Page Application (SPA)

Client-side rendering, communicating with the backend over REST.

Backend

Quarkus 3.37 REST API with Hexagonal Architecture, Jakarta Persistence (JPA), and Kafka event publishing.

Proposed System

1 Introduction & Proposed System

The application is built as three separated layers

Frontend

Angular Single Page Application (SPA)

Client-side rendering, communicating with the backend over REST.

Backend

Quarkus 3.37 REST API with Hexagonal Architecture, Jakarta Persistence (JPA), and Kafka event publishing.

Event-Driven

Incident events flow through Kafka to an independent analyzer that deduplicates and reports back asynchronously.

Table of Contents

2 Requirements Analysis

- ▶ Introduction & Proposed System
- ▶ **Requirements Analysis**
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ System Demonstration
- ▶ Conclusions & Future Work

A good SRS should be

Identifying FR and NFR

- Correct

A good SRS should be

Identifying FR and NFR

- Correct
- Unambiguous

A good SRS should be

Identifying FR and NFR

- Correct
- Unambiguous
- Complete

A good SRS should be

Identifying FR and NFR

- Correct
- Unambiguous
- Complete
- Consistent

A good SRS should be

Identifying FR and NFR

- Correct
- Unambiguous
- Complete
- Consistent
- Ranked (for importance or stability)

A good SRS should be

Identifying FR and NFR

- Correct
- Unambiguous
- Complete
- Consistent
- Ranked (for importance or stability)
- Verifiable

A good SRS should be

Identifying FR and NFR

- Correct
- Unambiguous
- Complete
- Consistent
- Ranked (for importance or stability)
- Verifiable
- Modifiable

A good SRS should be

Identifying FR and NFR

- Correct
- Unambiguous
- Complete
- Consistent
- Ranked (for importance or stability)
- Verifiable
- Modifiable
- Traceable

A good SRS should be

Identifying FR and NFR

- Correct
- Unambiguous
- Complete
- Consistent
- Ranked (for importance or stability)
- Verifiable
- Modifiable
- Traceable

Source: IEEE 830

RFC 2119 and ISO/IEC/IEEE 29148:2011

Identifying FR and NFR

- Each requirement is clearly identified.

RFC 2119 and ISO/IEC/IEEE 29148:2011

Identifying FR and NFR

- Each requirement is clearly identified.
- Based on **RFC 2119** keywords: **MUST**, **SHALL**, **SHOULD**, **MUST NOT**, etc.

RFC 2119 and ISO/IEC/IEEE 29148:2011

Identifying FR and NFR

- Each requirement is clearly identified.
- Based on **RFC 2119** keywords: **MUST**, **SHALL**, **SHOULD**, **MUST NOT**, etc.
- **ISO/IEC/IEEE 29148:2011**: engineering and managing requirements aligned with business goals.

Functional Requirements

Reporter & Department Admin

Reporter

- **FR6:** The system SHALL limit the incident list to show only the incidents reported by the acting user if their role is REPORTER.
- **FR11:** The system MUST NOT allow a REPORTER to assign or change the department of any incident.
- **FR9:** During incident creation, if the specified tag already exists, the system SHALL reuse it.

Functional Requirements

Reporter & Department Admin

Reporter

- **FR6:** The system SHALL limit the incident list to show only the incidents reported by the acting user if their role is REPORTER.
- **FR11:** The system MUST NOT allow a REPORTER to assign or change the department of any incident.
- **FR9:** During incident creation, if the specified tag already exists, the system SHALL reuse it.

Department Admin

- **FR13:** The system SHALL filter the listed incidents to only display those assigned to the acting user's department if the acting user is an active ADMIN.
- **FR14:** A department ADMIN SHALL only close incidents assigned to their own department.
- **FR23:** Once an ADMIN's role has expired, the system SHALL revoke their ability to close incidents and to reassign departments.

Functional Requirements

Super Admin & System / Analyzer

Super Admin

FR20: A SUPER_ADMIN SHALL be permitted to create a user of any type, including REPORTER, ADMIN, or another SUPER_ADMIN.

FR26: Any active ADMIN or SUPER_ADMIN SHALL be permitted to assign or reassign a department to an incident during the closure action, same as during a plain update.

FR28: A SUPER_ADMIN SHALL be able to create, update, and delete departments with unique names and descriptions.

Functional Requirements

Super Admin & System / Analyzer

Super Admin

- FR20:** A SUPER_ADMIN SHALL be permitted to create a user of any type, including REPORTER, ADMIN, or another SUPER_ADMIN.
- FR26:** Any active ADMIN or SUPER_ADMIN SHALL be permitted to assign or reassign a department to an incident during the closure action, same as during a plain update.
- FR28:** A SUPER_ADMIN SHALL be able to create, update, and delete departments with unique names and descriptions.

System / Analyzer

- **FR33:** The Analyzer SHALL detect potential duplicates among open incidents using fuzzy text similarity checking with a threshold > 0.4 .
- **NFR10:** Duplicate detection SHALL happen asynchronously without blocking the incident creation write path.

Note: Refer to the technical report for details on all 40 Functional Requirements.

Non-Functional Requirements

2 Requirements Analysis

Scalability & Elasticity

- **NFR1:** The api-service and analyzer-service SHALL be independently scalable via separate Kubernetes Deployments.

Note: Refer to the technical report for details on all 20 Non-Functional Requirements.

Non-Functional Requirements

2 Requirements Analysis

Scalability & Elasticity

- **NFR1:** The api-service and analyzer-service SHALL be independently scalable via separate Kubernetes Deployments.

Service Independence

- **NFR9:** Inter-service communication SHALL occur exclusively via Kafka topics (incident-created, incident-analysis-completed).

Note: Refer to the technical report for details on all 20 Non-Functional Requirements.

Non-Functional Requirements

2 Requirements Analysis

Scalability & Elasticity

- **NFR1:** The api-service and analyzer-service SHALL be independently scalable via separate Kubernetes Deployments.

Service Independence

- **NFR9:** Inter-service communication SHALL occur exclusively via Kafka topics (incident-created, incident-analysis-completed).

Transactional Integrity & Validation

- **NFR11:** All write operations (creation, update, closure, commenting) SHALL execute within a single @Transactional database transaction.

Note: Refer to the technical report for details on all 20 Non-Functional Requirements.

Non-Functional Requirements

2 Requirements Analysis

Scalability & Elasticity

- **NFR1:** The api-service and analyzer-service SHALL be independently scalable via separate Kubernetes Deployments.

Deployment & Containerization

- **NFR4:** All services (frontend, backend, analyzer, PostgreSQL, Kafka) SHALL be containerized and orchestrated via Docker Compose for local development.

Service Independence

- **NFR9:** Inter-service communication SHALL occur exclusively via Kafka topics (incident-created, incident-analysis-completed).

Transactional Integrity & Validation

- **NFR11:** All write operations (creation, update, closure, commenting) SHALL execute within a single @Transactional database transaction.

Note: Refer to the technical report for details on all 20 Non-Functional Requirements.

Non-Functional Requirements

2 Requirements Analysis

Scalability & Elasticity

- **NFR1:** The api-service and analyzer-service SHALL be independently scalable via separate Kubernetes Deployments.

Deployment & Containerization

- **NFR4:** All services (frontend, backend, analyzer, PostgreSQL, Kafka) SHALL be containerized and orchestrated via Docker Compose for local development.

Service Independence

- **NFR9:** Inter-service communication SHALL occur exclusively via Kafka topics (incident-created, incident-analysis-completed).

Testability & Security

- **NFR15:** Integration tests SHALL use Testcontainers (via Quarkus Dev Services) to verify persistence and REST endpoints against a real PostgreSQL instance.

Transactional Integrity & Validation

- **NFR11:** All write operations (creation, update, closure, commenting) SHALL execute within a single @Transactional database transaction.

Note: Refer to the technical report for details on all 20 Non-Functional Requirements.

Non-Functional Requirements

2 Requirements Analysis

Scalability & Elasticity

- **NFR1:** The api-service and analyzer-service SHALL be independently scalable via separate Kubernetes Deployments.

Deployment & Containerization

- **NFR4:** All services (frontend, backend, analyzer, PostgreSQL, Kafka) SHALL be containerized and orchestrated via Docker Compose for local development.

Service Independence

- **NFR9:** Inter-service communication SHALL occur exclusively via Kafka topics (incident-created, incident-analysis-completed).

Testability & Security

- **NFR15:** Integration tests SHALL use Testcontainers (via Quarkus Dev Services) to verify persistence and REST endpoints against a real PostgreSQL instance.

Transactional Integrity & Validation

- **NFR11:** All write operations (creation, update, closure, commenting) SHALL execute within a single @Transactional database transaction.

Note: Refer to the technical report for details on all 20 Non-Functional Requirements.

Table of Contents

3 Preliminary / Conceptual Design (The Dictionary)

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ System Demonstration
- ▶ Conclusions & Future Work

Conceptual Model

3 Preliminary / Conceptual Design (The Dictionary)

- **Stakeholder Language:** Technology-independent vocabulary of core entities.

Conceptual Model

3 Preliminary / Conceptual Design (The Dictionary)

- **Stakeholder Language:** Technology-independent vocabulary of core entities.
- **Intentional Omission:** No schemas, operations, or implementation details.

Conceptual Model

3 Preliminary / Conceptual Design (The Dictionary)

- **Stakeholder Language:** Technology-independent vocabulary of core entities.
- **Intentional Omission:** No schemas, operations, or implementation details.
- **Core Dictionary:** **User** (actor), **Incident** (tracked problem), **Department** (resolution owner), **Tag** (classification), **Comment** (collaboration thread).

Conceptual Model — Diagram

The dictionary above maps directly onto the entities and relationships below.

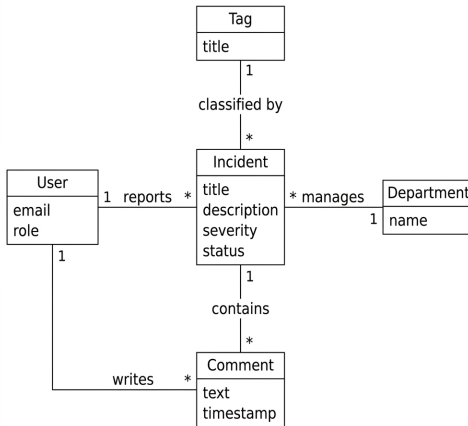


Table of Contents

4 Use Case Analysis

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ **Use Case Analysis**
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ System Demonstration
- ▶ Conclusions & Future Work

Use Case Overview

4 Use Case Analysis

- Each use case traces back to a functional requirement from the previous slide.

Use Case Overview

4 Use Case Analysis

- Each use case traces back to a functional requirement from the previous slide.
- The Analyzer is modeled as its own actor since it acts autonomously, off the back of an event, not a user request.

Use Case Overview

4 Use Case Analysis

- Each use case traces back to a functional requirement from the previous slide.
- The Analyzer is modeled as its own actor since it acts autonomously, off the back of an event, not a user request.
- <<extend>>: For optional steps (e.g. creating tags on the fly).

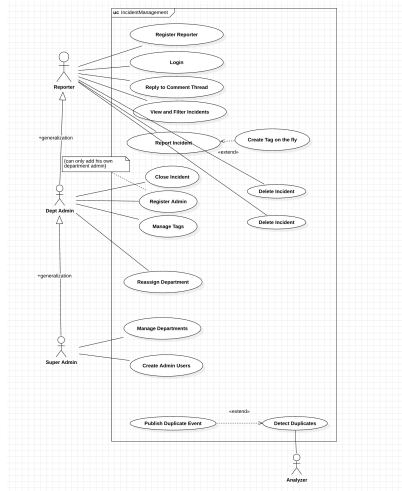
Use Case Overview

4 Use Case Analysis

- Each use case traces back to a functional requirement from the previous slide.
- The Analyzer is modeled as its own actor since it acts autonomously, off the back of an event, not a user request.
- <<extend>>: For optional steps (e.g. creating tags on the fly).
- <<include>>: For mandatory, shared steps.

Use Case Diagram

These actors and interactions are shown in the diagram below.



Detailed Use Case: Report Incident

4 Use Case Analysis

Actor	Reporter (inherited by Department Admin and Super Admin via generalization).
Level	User Goal.
Basic Course	1. The Actor selects the option to report a new incident. 2. The system presents the incident creation form. 3. The Actor fills in the title and severity (both mandatory), an optional description, and either picks an existing tag or types a new one. 4. The Actor submits the form. 5. The system validates the input (FR7), resolves the tag or creates it automatically if it doesn't exist yet (FR9, FR40), persists the incident with no department assigned (FR8), and returns the created incident to the Actor.
Postcondition	An incident-created event is published to Kafka, triggering the asynchronous duplicate-detection pipeline without blocking this use case's response.

Detailed Use Case: Reassign Department

4 Use Case Analysis

Actor	Department Admin or Super Admin.
Level	User Goal.
Precondition	The Actor is an active Admin; a Reporter can never perform this use case regardless of whose incident it is (FR11).
Basic Course	1. The Actor opens an incident they are permitted to update — their own department's incident, or (for a Super Admin) any incident. 2. The Actor selects a different Department from the dropdown, or clears it. 3. The Actor saves the change, either as a plain edit (FR17) or as part of closing the incident (FR26). 4. The system verifies the Actor's admin status, applies the new department assignment, and persists it.
Postcondition	If a Department Admin reassigned the incident away from their own department, it no longer appears in that Actor's own Retrieve Incidents result set on the next query. The same scoping rule applies uniformly.

More System Use Cases

4 Use Case Analysis

The remaining use cases are fully specified in the technical report:

- Retrieve Incidents

Refer to §4.1 (pages 10–18) of the technical report for complete specification tables.

More System Use Cases

4 Use Case Analysis

The remaining use cases are fully specified in the technical report:

- Retrieve Incidents
- Create Department

Refer to §4.1 (pages 10–18) of the technical report for complete specification tables.

More System Use Cases

4 Use Case Analysis

The remaining use cases are fully specified in the technical report:

- Retrieve Incidents
- Create Department
- Add Comment

Refer to §4.1 (pages 10–18) of the technical report for complete specification tables.

More System Use Cases

4 Use Case Analysis

The remaining use cases are fully specified in the technical report:

- Retrieve Incidents
- Create Department
- Add Comment
- Close Incident

Refer to §4.1 (pages 10–18) of the technical report for complete specification tables.

More System Use Cases

4 Use Case Analysis

The remaining use cases are fully specified in the technical report:

- Retrieve Incidents
- Create Department
- Add Comment
- Close Incident
- User Administration

Refer to §4.1 (pages 10–18) of the technical report for complete specification tables.

More System Use Cases

4 Use Case Analysis

The remaining use cases are fully specified in the technical report:

- Retrieve Incidents
- Create Department
- Add Comment
- Close Incident
- User Administration
- Authentication

Refer to §4.1 (pages 10–18) of the technical report for complete specification tables.

More System Use Cases

4 Use Case Analysis

The remaining use cases are fully specified in the technical report:

- Retrieve Incidents
- Create Department
- Add Comment
- Close Incident
- User Administration
- Authentication
- Incident Deletion

Refer to §4.1 (pages 10–18) of the technical report for complete specification tables.

Temporal Constraints & Expiration Rules

4 Use Case Analysis

- **adminExpiresAt:** Admin actions require current time $<$ expiration timestamp (or null).

Temporal Constraints & Expiration Rules

4 Use Case Analysis

- **adminExpiresAt:** Admin actions require current time $<$ expiration timestamp (or null).
- **Graceful Degradation:** Past expiration, the user's admin state is automatically revoked.

Temporal Constraints & Expiration Rules

4 Use Case Analysis

- **adminExpiresAt:** Admin actions require current time $<$ expiration timestamp (or null).
- **Graceful Degradation:** Past expiration, the user's admin state is automatically revoked.
- **Behavioral Shift:** Expired admins fall back to Reporter-level permissions.

Table of Contents

5 Domain Modeling

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ **Domain Modeling**
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ System Demonstration
- ▶ Conclusions & Future Work

Domain Model

5 Domain Modeling

- **Unidirectional Associations:** Child entities reference their parents only (e.g. Incident → User, Department, Tag) — no back-references.

Domain Model

5 Domain Modeling

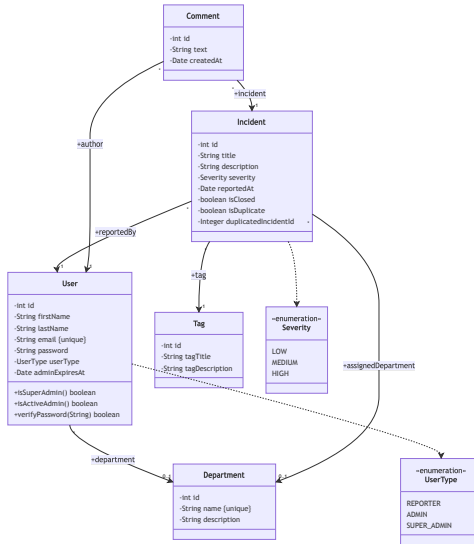
- **Unidirectional Associations:** Child entities reference their parents only (e.g. Incident → User, Department, Tag) — no back-references.
- **Incident → Tag:** Exactly one Tag per Incident — deliberate simplification kept from the pre-existing codebase.

Domain Model

5 Domain Modeling

- **Unidirectional Associations:** Child entities reference their parents only (e.g. Incident → User, Department, Tag) — no back-references.
- **Incident → Tag:** Exactly one Tag per Incident — deliberate simplification kept from the pre-existing codebase.
- **Comment:** First-class entity, not a text field — powers both comments and closing commentary.

Domain Model — Class Diagram



Domain Model: Course Design Idioms

5 Domain Modeling

- **Roles > Subclasses:** UserType enum instead of subclassing User into Reporter/Admin/SuperAdmin.

Domain Model: Course Design Idioms

5 Domain Modeling

- **Roles > Subclasses:** UserType enum instead of subclassing User into Reporter/Admin/SuperAdmin.
- **No Target/Agent Anti-Pattern:** Invariants live inside target entities (User, Incident); services handle orchestration only.

Domain Model: Course Design Idioms

5 Domain Modeling

- **Roles > Subclasses:** UserType enum instead of subclassing User into Reporter/Admin/SuperAdmin.
- **No Target/Agent Anti-Pattern:** Invariants live inside target entities (User, Incident); services handle orchestration only.

Invariant Enforcement

User:

- BCrypt hashing & password validation.
- No-whitespace names; email must have @ and ..

Incident:

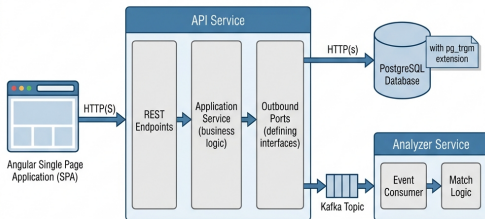
- Non-empty title/severity required.
- Collapses whitespace and trims.

Table of Contents

6 Application Architecture

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ **Application Architecture**
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ System Demonstration
- ▶ Conclusions & Future Work

Application Architecture (Hexagonal)



Flow: Angular SPA → REST Endpoints → Application Service → Outbound Ports → Kafka Topic / PostgreSQL. The Analyzer Service reacts independently, consuming from Kafka on its own.

Application Architecture (Hexagonal)

6 Application Architecture

- **Hexagonal:** Domain behind ports; adapters for HTTP, JPA, and Kafka.

Application Architecture (Hexagonal)

6 Application Architecture

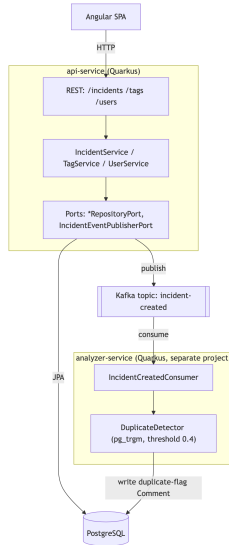
- **Hexagonal:** Domain behind ports; adapters for HTTP, JPA, and Kafka.
- **Partitioned:** API and Analyzer services share zero Java code.

Application Architecture (Hexagonal)

6 Application Architecture

- **Hexagonal:** Domain behind ports; adapters for HTTP, JPA, and Kafka.
- **Partitioned:** API and Analyzer services share zero Java code.
- **Clean Boundaries:** Inward flow — services depend only on port interfaces.

System Architecture & Data Flow



System Architecture & Data Flow

6 Application Architecture

- **Client:** Angular frontend talks to Quarkus API over REST.

System Architecture & Data Flow

6 Application Architecture

- **Client:** Angular frontend talks to Quarkus API over REST.
- **Persistence:** API service handles transactions and persists to PostgreSQL.

System Architecture & Data Flow

6 Application Architecture

- **Client:** Angular frontend talks to Quarkus API over REST.
- **Persistence:** API service handles transactions and persists to PostgreSQL.
- **Async Pipeline:** Analyzer microservice consumes events via Kafka.

Table of Contents

7 Detailed Design & Main Components

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ System Demonstration
- ▶ Conclusions & Future Work

Detailed Design & Main Components

7 Detailed Design & Main Components

Package Tree

```
.../incident/  
|-- domain/  
|   |-- Incident, Tag, User, ...  
|-- application/  
|   |-- port/out/  
|       |-- IncidentRepositoryPort  
|       |-- IncidentEventPublisherPort  
|   |-- service/  
|       |-- IncidentService  
|-- adapters/  
|   |-- in/rest/  
|       |-- IncidentResource  
|   |-- out/persistence/  
|       |-- IncidentPostgresRepository  
|   |-- out/messaging/  
|       |-- KafkaIncidentEventPublisher
```

CDI & JPA Wiring

```
// domain (persistence)  
@Entity @Table(name = "incidents")  
public class Incident {  
    @Id @GeneratedValue private int id;  
}
```

- **Logical Bridge:** Connects the hexagonal core to Quarkus, Angular, and Hibernate/JPA.
- **Pragmatic Hexagonal:** Domain entities carry @Entity directly — no separate persistence model.
- **Relational Mapping:** Unidirectional associations map to @ManyToOne foreign keys (e.g. Incident → Tag).
- **CDI Auto-Wiring:** The one @ApplicationScoped bean implementing IncidentRepositoryPort is injected automatically — no qualifiers needed.

Detailed Design & Main Components

7 Detailed Design & Main Components

Package Tree

```
.../incident/  
|-- domain/  
|   |-- Incident, Tag, User, ...  
|-- application/  
|   |-- port/out/  
|   |   |-- IncidentRepositoryPort  
|   |   |-- IncidentEventPublisherPort  
|   |-- service/  
|   |   |-- IncidentService  
|-- adapters/  
|   |-- in/rest/  
|   |   |-- IncidentResource  
|   |-- out/persistence/  
|   |   |-- IncidentPostgresRepository  
|   |-- out/messaging/  
|   |   |-- KafkaIncidentEventPublisher
```

CDI & JPA Wiring

```
// domain (persistence)  
@Entity @Table(name = "incidents")  
public class Incident {  
    @Id @GeneratedValue private int id;  
}  
  
// application.service  
@ApplicationScoped  
public class IncidentService {  
    @Inject  
    IncidentRepositoryPort incidentRepository;  
}
```

- **Logical Bridge:** Connects the hexagonal core to Quarkus, Angular, and Hibernate/JPA.
- **Pragmatic Hexagonal:** Domain entities carry `@Entity` directly — no separate persistence model.
- **Relational Mapping:** Unidirectional associations map to `@ManyToOne` foreign keys (e.g. Incident → Tag).
- **CDI Auto-Wiring:** The one `@ApplicationScoped` bean implementing `IncidentRepositoryPort` is injected automatically — no qualifiers needed.

Detailed Design & Main Components

7 Detailed Design & Main Components

Package Tree

```
.../incident/  
|-- domain/  
|   |-- Incident, Tag, User, ...  
|-- application/  
|   |-- port/out/  
|       |-- IncidentRepositoryPort  
|       |-- IncidentEventPublisherPort  
|   |-- service/  
|       |-- IncidentService  
|-- adapters/  
|   |-- in/rest/  
|       |-- IncidentResource  
|   |-- out/persistence/  
|       |-- IncidentPostgresRepository  
|   |-- out/messaging/  
|       |-- KafkaIncidentEventPublisher
```

CDI & JPA Wiring

```
// domain (persistence)  
@Entity @Table(name = "incidents")  
public class Incident {  
    @Id @GeneratedValue private int id;  
}  
  
// application.service  
@ApplicationScoped  
public class IncidentService {  
    @Inject  
    IncidentRepositoryPort incidentRepository;  
}  
  
// adapters.out.persistence  
@ApplicationScoped  
public class IncidentPostgresRepository  
    implements IncidentRepositoryPort {  
    @Override  
    public void save(Incident incident) {  
        em.persist(incident);  
    }  
}
```

- **Logical Bridge:** Connects the hexagonal core to Quarkus, Angular, and Hibernate/JPA.
- **Pragmatic Hexagonal:** Domain entities carry @Entity directly — no separate persistence model.
- **Relational Mapping:** Unidirectional associations map to @ManyToOne foreign keys (e.g. Incident → Tag).
- **CDI Auto-Wiring:** The one @ApplicationScoped bean implementing IncidentRepositoryPort is injected automatically — no qualifiers needed.

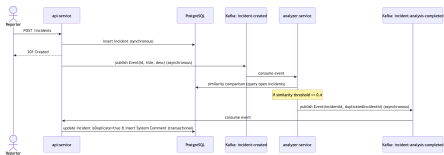
Duplicate Detection — the EDA Deep Dive

7 Detailed Design & Main Components

1. JAX-RS POST (Resource)

```

// Java API for RESTful Web Services
// adapters.in.rest
@POST
public Incident create(CreateIncidentRequest request) {
    return incidentService.create(request.title(), ...);
}
  
```



Duplicate Detection — the EDA Deep Dive

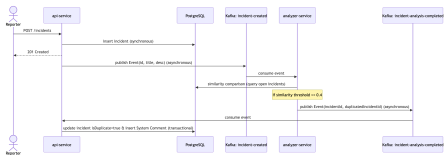
7 Detailed Design & Main Components

1. JAX-RS POST (Resource)

```
// Java API for RESTful Web Services
// adapters.in.rest
@POST
public Incident create(CreateIncidentRequest request) {
    return incidentService.create(request.title(), ...);
}
```

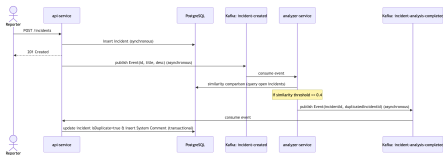
2. Service & DB Save (JPA)

```
// business logic + persistence
// application.service
// adapters.out.persistence
@Transactional
public Incident create(...) {
    incidentRepository.save(incident); // -> persistence
    eventPublisher.publishIncidentCreated(...); // -> step 3
}
```



Duplicate Detection — the EDA Deep Dive

7 Detailed Design & Main Components



1. JAX-RS POST (Resource)

```
// Java API for RESTful Web Services
// adapters.in.rest
@POST
public Incident create(CreateIncidentRequest request) {
    return incidentService.create(request.title(), ...);
}
```

2. Service & DB Save (JPA)

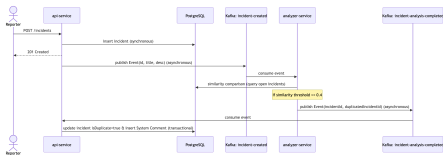
```
// business logic + persistence
// application.service
// adapters.out.persistence
@Transactional
public Incident create(...) {
    incidentRepository.save(incident); // -> persistence
    eventPublisher.publishIncidentCreated(...); // -> step 3
}
```

3. Outbound Adapter (Kafka)

```
// publish to Kafka
// adapters.out.messaging
@Inject @Channel('`incident-created`) Emitter<String> emitter;
public void publishIncidentCreated(...) {
    emitter.send(toJson(event));
}
```

Duplicate Detection — the EDA Deep Dive

7 Detailed Design & Main Components



1. JAX-RS POST (Resource)

```
// Java API for RESTful Web Services
// adapters.in.rest
@POST
public Incident create(CreateIncidentRequest request) {
    return incidentService.create(request.title(), ...);
}
```

2. Service & DB Save (JPA)

```
// business logic + persistence
// application.service
// adapters.out.persistence
@Transactional
public Incident create(...) {
    incidentRepository.save(incident); // -> persistence
    eventPublisher.publishIncidentCreated(...); // -> step 3
}
```

3. Outbound Adapter (Kafka)

```
// publish to Kafka
// adapters.out.messaging
@Inject @Channel('`incident-created`) Emitter<String> emitter;
public void publishIncidentCreated(...) {
    emitter.send(toJson(event));
}
```

4. Inbound Consumer (Analyzer)

```
// consume from Kafka (separate microservice)
// analyzer.consumer
@Inject @Channel('`incident-created`)
public void consume(String json) {
    duplicateDetector.checkForDuplicate(...);
}
```


CQRS-Lite: Flexible Reads

7 Detailed Design & Main Components

- **Dynamic Read Filters:** Single query endpoint supports optional filter parameters: GET /incidents?tag=&severity=&status=.
- **Role-Scoped Narrowing:** Dynamic results are bounded by user permissions:
 - **Super Admin:** Sees all incidents.
 - **Dept Admin:** Filtered to their department.
 - **Reporter:** Filtered to own reports.

CQRS-lite

Isolates read query logic from command writes, without introducing separate query tables or event sourcing.

Dynamic JPQL

```
// adapters.out.persistence
StringBuilder jpql = new StringBuilder(
    "select i from Incident i where 1=1");
if (tag != null)
    jpql.append(" and i.tag.tagTitle = :tag");
if (severity != null)
    jpql.append(" and i.severity = :severity");
// ...one optional clause per filter
em.createQuery(jpql.toString(), Incident.class)
    .getResultList();
```

Table of Contents

8 Database Design

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ **Database Design**
- ▶ Implementation & Deployment
- ▶ System Demonstration
- ▶ Conclusions & Future Work

Database Design

8 Database Design

- **Hibernate-Managed Schema:** Tables auto-generated from JPA annotations.

Database Design

8 Database Design

- **Hibernate-Managed Schema:** Tables auto-generated from JPA annotations.
- **Incident as Hub:** FKs to User, Tag, and optionally Department.

Database Design

8 Database Design

- **Hibernate-Managed Schema:** Tables auto-generated from JPA annotations.
- **Incident as Hub:** FKs to User, Tag, and optionally Department.
- **Comment Threading:** Each comment FKs to its Incident and author User.

Database Design

8 Database Design

- **Hibernate-Managed Schema:** Tables auto-generated from JPA annotations.
- **Incident as Hub:** FKs to User, Tag, and optionally Department.
- **Comment Threading:** Each comment FKs to its Incident and author User.
- **Unique Indexes:** `departments.name` and `users.email`.

Entity-Relationship Diagram

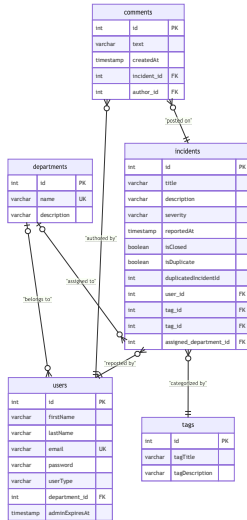


Table of Contents

9 Implementation & Deployment

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment**
- ▶ System Demonstration
- ▶ Conclusions & Future Work

Implementation & Deployment

9 Implementation & Deployment

Backend API

- Quarkus 3.37.1 (Java 17)
- Jakarta REST, CDI, Hibernate ORM
- Maven-based

Frontend Client

- Angular 19 standalone components
- Functional Auth/Admin Guards
- Dark-theme dashboard

Database & Messaging

- PostgreSQL 16 (pg_trgm)
- Kafka (KRaft mode)
- SmallRye Reactive Messaging

Orchestration

- Local: Docker Compose
- Production: Kubernetes (Minikube)
- Horizontal Pod Autoscaler (HPA)

Local Orchestration (Docker Compose)

9 Implementation & Deployment

- **Single Command Bootstrapping:** Spin up the entire environment (API, Analyzer, PostgreSQL, Kafka, and Angular Frontend) via `docker compose up`.

Compose Topology

- `frontend`: Angular 19 Client
- `api-service`: Backend endpoint provider
- `analyzer-service`: Duplicate check listener
- `postgres`: Relational storage DB
- `kafka`: Broker (incident topics)

Local Orchestration (Docker Compose)

9 Implementation & Deployment

- **Single Command Bootstrapping:** Spin up the entire environment (API, Analyzer, PostgreSQL, Kafka, and Angular Frontend) via `docker compose up`.
- **Decoupled Execution:** Containerization ensures uniform execution and isolated networks between independent services.

Compose Topology

- `frontend`: Angular 19 Client
- `api-service`: Backend endpoint provider
- `analyzer-service`: Duplicate check listener
- `postgres`: Relational storage DB
- `kafka`: Broker (incident topics)

Local Orchestration (Docker Compose)

9 Implementation & Deployment

- **Single Command Bootstrapping:** Spin up the entire environment (API, Analyzer, PostgreSQL, Kafka, and Angular Frontend) via `docker compose up`.
- **Decoupled Execution:** Containerization ensures uniform execution and isolated networks between independent services.
- **Profile-Aware Config:** Integrates seamlessly with profile overrides (e.g. `%dev` configuration profiles) for local testing.

Compose Topology

- `frontend`: Angular 19 Client
- `api-service`: Backend endpoint provider
- `analyzer-service`: Duplicate check listener
- `postgres`: Relational storage DB
- `kafka`: Broker (incident topics)

Production Deployment Model (Kubernetes)

9 Implementation & Deployment

- **Namespace Isolation:** Services are packaged in declarative manifests within a dedicated namespace.

Production Deployment Model (Kubernetes)

9 Implementation & Deployment

- **Namespace Isolation:** Services are packaged in declarative manifests within a dedicated namespace.
- **Target Scalability:** Scaling is governed by standard Horizontal Pod Autoscaling (HPA) targeting 50% CPU load.

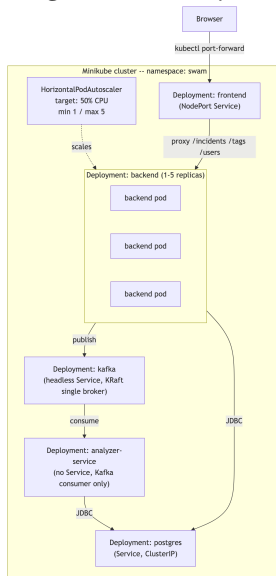
Production Deployment Model (Kubernetes)

9 Implementation & Deployment

- **Namespace Isolation:** Services are packaged in declarative manifests within a dedicated namespace.
- **Target Scalability:** Scaling is governed by standard Horizontal Pod Autoscaling (HPA) targeting 50% CPU load.
- **Service Abstraction:** Exposes standard ClusterIP and NodePort configurations for internal microservice and external ingress routing.

Production Deployment — Diagram

Namespace isolation, HPA scaling, and service exposure shown below.



Integration Testing & API Safeguards

9 Implementation & Deployment

- **REST-Assured Safeguards:** Validates API access policies, input restrictions, and role constraints dynamically.

Integration Testing & API Safeguards

9 Implementation & Deployment

- **REST-Assured Safeguards:** Validates API access policies, input restrictions, and role constraints dynamically.
- **Deletion Constraints:** Asserts that only the reporting user can delete their own ticket (returning 204 No Content).

REST Deletion Assertion

```
given()
  .when()
  .delete(`/incidents/${id} + id + `?actingUserId=${reporterId}`)
  .then()
  .statusCode(204);
```

Integration Testing & API Safeguards

9 Implementation & Deployment

- **REST-Assured Safeguards:** Validates API access policies, input restrictions, and role constraints dynamically.
- **Deletion Constraints:** Asserts that only the reporting user can delete their own ticket (returning 204 No Content).
- **Authorization Bounds:** Unauthorized attempts by another user return 400 Bad Request.

REST Deletion Assertion

```
given()
  .when()
  .delete(`/incidents/${id} + ``?actingUserId=${reporterId}``)
  .then()
  .statusCode(204);
```

Hijack Attempt Safeguard

```
given().contentType(`application/json`)
  .body(`{"actingUserId": "${otherId} + ``...}``)
  .when().put(`/incidents/${id}``)
  .then().statusCode(400); // Bad Request
```

Table of Contents

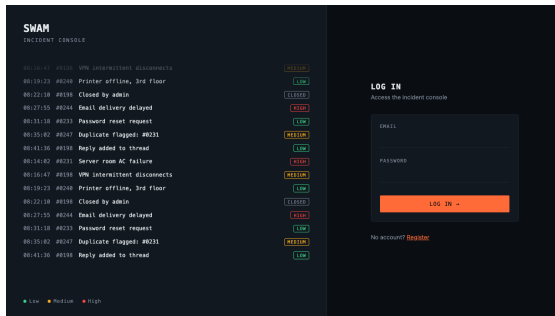
10 System Demonstration

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ **System Demonstration**
- ▶ Conclusions & Future Work

Demo: Login Interface

10 System Demonstration

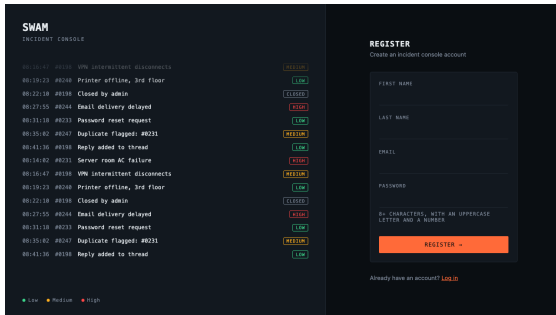
Secures application access, verifying credentials against bcrypt-hashed database values. Expired admin status is automatically calculated upon login.



Demo: User Registration

10 System Demonstration

Allows new users to create accounts.
Enforces input validation rules, such
as mandatory fields and a minimum
8-character password length
containing a digit and uppercase
letter.



The screenshot displays two side-by-side panels from the SWAM system interface.

Left Panel: SWAM INCIDENT CONSOLE

This panel shows a list of incidents with their timestamps, IDs, descriptions, and severity levels. The incidents are as follows:

Timestamp	ID	Description	Severity
00:10:47	#0100	VNM intermittent disconnects	MEDIUM
00:19:23	#0200	Printer offline, 3rd floor	LOW
00:22:10	#0100	Closed by admin	CLOSED
00:27:55	#0204	Email delivery delayed	HIGH
00:31:10	#0233	Password reset request	LOW
00:35:02	#0247	Duplicate flagged: #0231	MEDIUM
00:41:36	#0100	Reply added to thread	LOW
00:14:02	#0231	Server room AC failure	HIGH
00:16:47	#0100	VNM intermittent disconnects	MEDIUM
00:19:23	#0200	Printer offline, 3rd floor	LOW
00:22:10	#0100	Closed by admin	CLOSED
00:27:55	#0204	Email delivery delayed	HIGH
00:31:10	#0233	Password reset request	LOW
00:35:02	#0247	Duplicate flagged: #0231	MEDIUM
00:41:36	#0100	Reply added to thread	LOW

A legend at the bottom indicates the severity levels: Low (green dot), Medium (yellow dot), and High (red dot).

Right Panel: REGISTER

This panel is titled "REGISTER" and includes the instruction "Create an incident console account". It contains a form with the following fields:

- FIRST NAME
- LAST NAME
- EMAIL
- PASSWORD

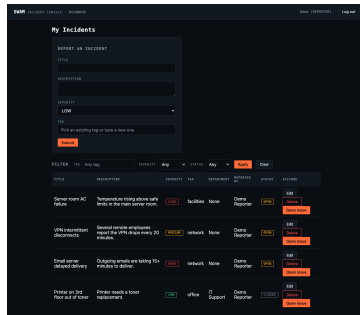
Below the password field, there is a requirement: "8+ CHARACTERS, WITH AN UPPERCASE LETTER AND A NUMBER". At the bottom of the form is an orange button labeled "REGISTER =>".

At the very bottom of the right panel, there is a link: "Already have an account? [Log in](#)".

Demo: Reporter View

10 System Demonstration

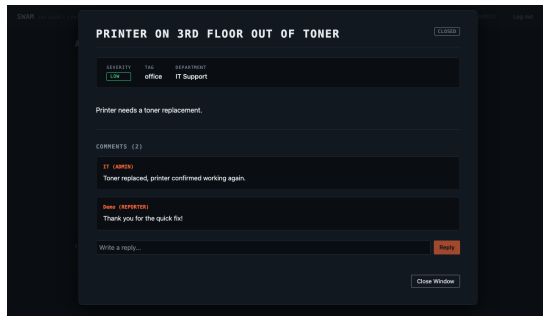
A logged-in Reporter sees only their own reported incidents. The dashboard supports tag suggestions, details, and collaborative comment threads.



Demo: Admin View — Incident & Comments

10 System Demonstration

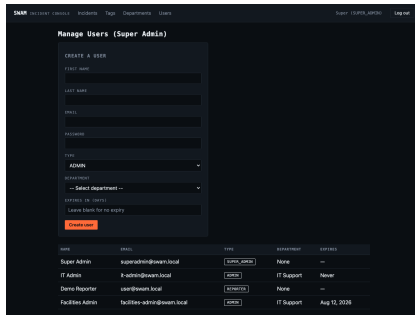
Admins view incidents assigned to their specific department. The detailed view displays the full collaborative comment thread and closing comment.



Demo: User Administration

10 System Demonstration

Super Admins can view all registered users and assign role types (Reporter, Admin, Super Admin) or set optional admin expiration dates.

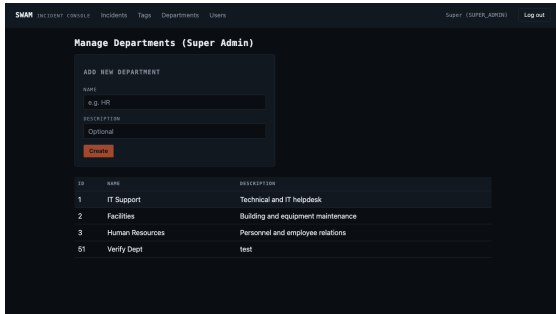


NAME	EMAIL	TYPE	DEPARTMENT	EXPIRES
Super Admin	superadmin@swan.local	SUPER_ADMIN	None	—
IT Admin	it-admin@swan.local	ADMIN	IT Support	Never
Demo Reporter	user@swan.local	REPORTER	None	—
Facilities Admin	facilities-admin@swan.local	ADMIN	IT Support	Aug 12, 2026

Demo: Department Administration

10 System Demonstration

Enables Super Admins to create, update, and manage unique departments with descriptions, facilitating structured incident routing.



SWAN INCIDENT CONSOLE Incidents Tags Departments Users Super (SUPER_ADMIN) Log out

Manage Departments (Super Admin)

ADD NEW DEPARTMENT

NAME
e.g. HR

DESCRIPTION
Optional

Create

ID	NAME	DESCRIPTION
1	IT Support	Technical and IT helpdesk
2	Facilities	Building and equipment maintenance
3	Human Resources	Personnel and employee relations
51	Verify Dept	test

Demo: Tags Management

10 System Demonstration

Gated by route protectors, this interface lets admins add, rename, or delete category tags dynamically suggested during incident creation.

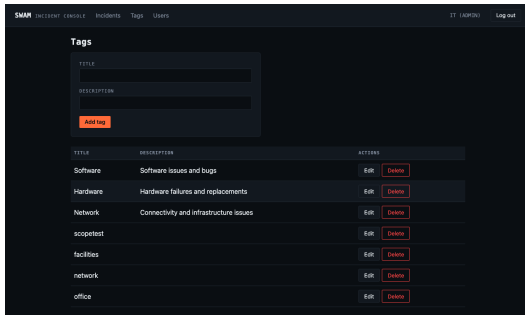


Table of Contents

11 Conclusions & Future Work

- ▶ Introduction & Proposed System
- ▶ Requirements Analysis
- ▶ Preliminary / Conceptual Design (The Dictionary)
- ▶ Use Case Analysis
- ▶ Domain Modeling
- ▶ Application Architecture
- ▶ Detailed Design & Main Components
- ▶ Database Design
- ▶ Implementation & Deployment
- ▶ System Demonstration
- ▶ **Conclusions & Future Work**

Conclusions & Future Developments

11 Conclusions & Future Work

- **Delivered:** Functional Hexagonal Architecture, asynchronous Event-Driven pipeline, lightweight CQRS reads, and elastic Kubernetes deployment.

Conclusions & Future Developments

11 Conclusions & Future Work

- **Delivered:** Functional Hexagonal Architecture, asynchronous Event-Driven pipeline, lightweight CQRS reads, and elastic Kubernetes deployment.
- **Future Work:**

Conclusions & Future Developments

11 Conclusions & Future Work

- **Delivered:** Functional Hexagonal Architecture, asynchronous Event-Driven pipeline, lightweight CQRS reads, and elastic Kubernetes deployment.
- **Future Work:**
 - *Stage-2 Semantic Deduplication:* pgvector and sentence embeddings (already implemented on separate branch).

Conclusions & Future Developments

11 Conclusions & Future Work

- **Delivered:** Functional Hexagonal Architecture, asynchronous Event-Driven pipeline, lightweight CQRS reads, and elastic Kubernetes deployment.
- **Future Work:**
 - *Stage-2 Semantic Deduplication:* pgvector and sentence embeddings (already implemented on separate branch).
 - *Transactional Outbox Pattern:* removes the current dual-write risk between the incident save and the Kafka publish.

Conclusions & Future Developments

11 Conclusions & Future Work

- **Delivered:** Functional Hexagonal Architecture, asynchronous Event-Driven pipeline, lightweight CQRS reads, and elastic Kubernetes deployment.
- **Future Work:**
 - *Stage-2 Semantic Deduplication:* pgvector and sentence embeddings (already implemented on separate branch).
 - *Transactional Outbox Pattern:* removes the current dual-write risk between the incident save and the Kafka publish.
 - *Real JWT Authentication:* replaces the current stub UserType role.

Acknowledgments

11 Conclusions & Future Work

- **AI-Assisted Development (Claude, Anthropic):**
 - Writing IT and unit tests to validate functional and non-functional requirements.

SWAM: Event-Driven Incident Management System

Thank you for listening!
Any questions?